

CSEN604: Data Bases II

Mohamed Karam Gabr

Introduction

Spring 2025

CSEN604: Data Bases II

- Instructor
 - E-mail: mohamed.kamel-gabr@guc.edu.eg
 - Office: C7.213
 - TAs:
 - Mahinar Mohamed
 - Ahmed Hassan
 - Eslam Sabry
 - Ahmed Nagy
 - Mostafa Ashraf
 - Fatma Alaa
- Textbook
 - Database Systems: The Complete Book (2nd Edition)
ISBN-13: 978-0131873254
 - Database System Concepts (7th Edition)
ISBN 978-0078022159

CSEN604: Data Bases II

- Course Evaluation
 - Quizzes (Best 2 of 3): 15%
 - Mid-term exam: 25%
 - Project: 20%
 - Final exam: 40%

Acknowledgment

- These slides are based on the course of CSEN 604 Data Bases II taught by Dr. Wael Abouelsaadat that are based on the book of Database Systems; the Complete Book.
- And on the book Database System Concepts: Seventh Edition, and its slides.

CSEN604: Data Bases II

- Topics to be covered:
 - Conventional Indices.
 - B+ Trees.
 - Hash Indices.
 - Multidimensional Indices.
 - Query Optimization I.
 - Query Optimization II.
 - Query Optimization III.
 - Logging and Recovery.
 - Transactions I.
 - Transactions II.

Course Objectives

- Internals of RDBMS
- Algorithms & Data Structures related to RDBMS

select * from
table where
value=x;



ResultSet

--	--	--

Misusing Database Engines

```
CREATE TABLE Customer (First_Name char(50),  
                        Last_Name char(50),  
                        Address char(50),  
                        City char(50),  
                        Country char(25),  
                        Birth_Date date);
```

```
// do some inserts..
```

```
select * from Customer where Address = '' ;   $O(n)$ 
```

```
select * from Customer where first_name = '' ;   $O(n)$ 
```

```
select * from Customer where Country = '' ;   $O(n)$ 
```

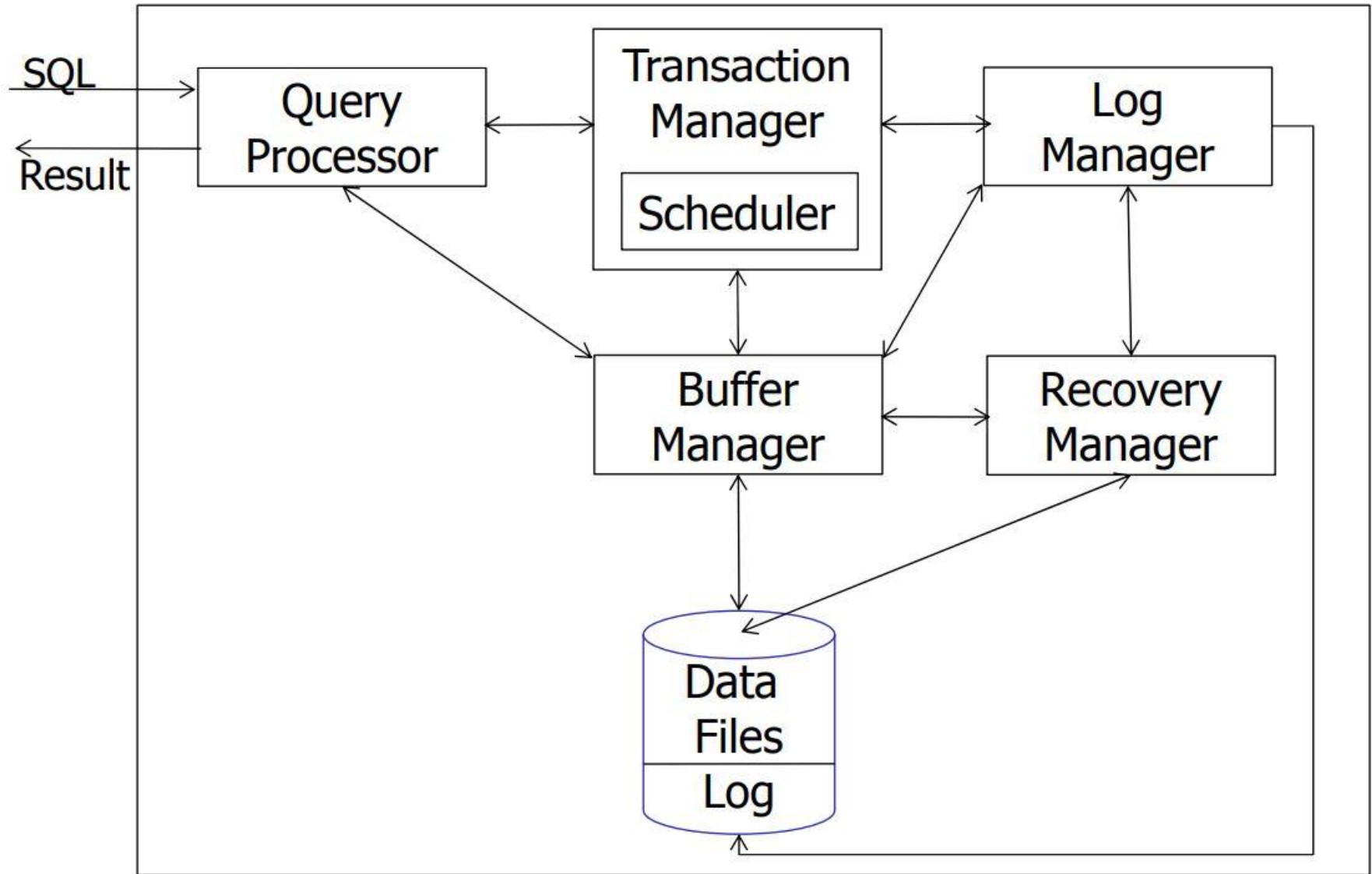
How to fix this?

Using Index

What's an Index

- An index is a data structure built on one or more columns to enhance the performance of SQL queries on those columns.
- Without an index, the database engine would be doing linear search in the data $O(n)$ to answer a query.
- Using an index, the performance is typically enhanced to $O(\log n)$ or $O(1)$.

RDBMS Architecture



Memory Types

- The computer contains two memory types: Hard Disk (HDD, SSD), and RAM.
- Disk access is the most expensive thing databases do.
- Disk IO is usually unavoidable.
- Usually, the data simply does not fit into memory.
- Data needs to be persistent when the DB is shut down (server crashes).
- DBs work very hard to minimize the amount of disk IO.
- Sectors are the unit of reading and writing from the disk.
- **Blocks** are logical units of storage used by an application such as a DBMS.
- **Blocks** typically consist of several sectors.

Pages and Blocks

- A page is a fixed-size block of data.
 - It can contain tuples, meta-data, indexes, log records.
- Each page is given a unique identifier.
 - The DBMS uses an indirection layer to map page ids to physical locations.
- Historically a page ranged from 16KB to 64KB.
- What you can read without moving the head on the HD.

Organization of Data in Files on disk

- Goals:
 - Allow insertion/deletions of tuples/records.
 - Fetch a particular record (specified by record id).
 - Find all tuples that match a condition (say SSN = 123) ?
- Simplest case:
 - A relation is mapped to a file.
 - A file contains a sequence of records.
 - Each record corresponds to a logical tuple.
- More Sophisticated approach:
 - A relation is mapped to several files (blocks).
 - A block contains a sequence of records.

How are tuples/records stored within a block

Fixed-Length Records:

- Store record i starting from byte $n * (i - 1)$, where n is the size of each record.
- Record access is simple, but records may cross blocks

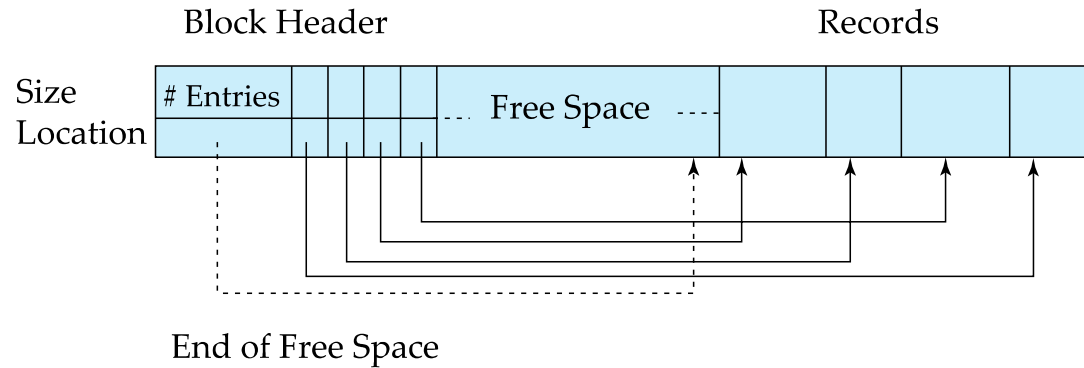
$$n = \underbrace{l_1}_{\text{A-102}} + \underbrace{l_2}_{\text{Perryridge}} + \underbrace{l_3}_{\text{400}}$$

- Cons:
 - Wasted space.
- Insertion:
 - Simply append at the end of the record.
- Deletion:
 - Rearrange (defrag)

record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 2	A-215	Mianus	700
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600
record 8	A-218	Perryridge	700

How are tuples/records stored within a block

Variable-Length Records:



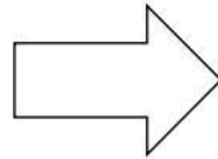
- Divide storage into units/pages of equal size (First slotted page structure).
- Store a record inside a page.
- Headers are used as a indirection mechanism.
- **Header contains:**
 - Number of record entries.
 - End of free space in the block location and size of each record.
 - Pointers should not point directly to record instead they should point to the entry for the record in header.
- Advantage:
 - Pages grow from right to left.
 - The records may move inside the page but entry in the header must be updated, but the outside world is oblivious to it.

Sorted Sequential FO

- Suitable for applications that require sequential processing of the entire file
- The records in the file are ordered by a **search-key**



record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 2	A-215	Mianus	700
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600
record 8	A-218	Perryridge	700



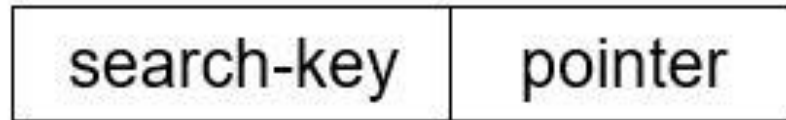
		350
		400
		500
		600
		700
		700
		700
		750
		900

Sorted Sequential FO

- Deletion:
 - use pointer chains.
 - Can become ***fragmented***, so must reorganize every once in a while.
- Insertion:
 - locate the position where the record is to be inserted.
 - if there is free space insert there.
 - if no free space, insert the record in an **overflow block**.
 - In either case, pointer chain must be updated.
- Need to reorganize the file from time to time to restore sequential order.
- For a specific column retrieval, use ***Indexing***.

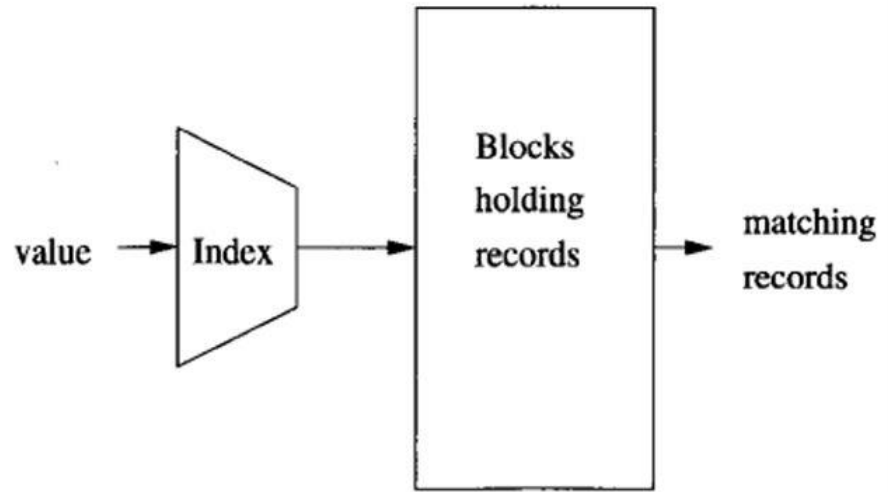
Basic Concepts

- Indexing mechanisms used to speed up access to desired data.
- **Search Key** - attribute to set of attributes used to look up records in a file.
- An **index file** consists of records (called **index entries**) of the form



- Index files are typically much smaller than the original file
- Two basic kinds of indices:
 - **Ordered indices:** search keys are stored in sorted order
 - **Hash indices:** search keys are distributed uniformly across “buckets” using a “hash function”.

Why Use Index



- A single-level index is an auxiliary file that makes it more efficient to search for a record in the data file
- An index has to be sorted according to a certain indexing field.
- Index structures provide secondary access paths.
- Any field can be used as an index.
- Multiple indices can be constructed.
- Index files are small and can be retrieved with less block accesses.

Key vs. Index vs. Clustering

- **Key:** logical concept.
- **Index:** a secondary storage data structure.
- **Clustering:** disk/page organizational issue According to which column the data is stored in pages.
- The clustering (i.e. sorting) column does not have to be the key column. Any column could be used in clustering the data in the pages of the table.